

FASTR Analytics Platform – Security Documentation

Authentication & Authorisation

Authentication is handled by Clerk, integrated into the Hono backend via `@hono/clerk-auth` middleware and applied globally to all routes. User identity is verified via session claims before any route handler executes. Authorisation uses a two-tier role-based access control system: instance-level permissions (e.g. can view data, can configure data) are enforced via a global permission middleware, while project-level permissions are enforced separately using a `Project-Id` request header matched against per-project user roles in the database. A set of trusted global admins are given all permissions for the purposes of site deployment, configuration, and management.. Auth bypass is available for local development but is explicitly disabled in production.

Data Isolation Across Countries

Each country runs as a fully isolated deployment, with its own dedicated PostgreSQL, Valkey, and application containers on its own private Docker network, and its own directory on the block storage volume. There is no shared database, shared cache, or shared filesystem path between countries, so cross-country leakage or interference is structurally impossible.

Data Isolation Across Projects

Each project is backed by its own isolated PostgreSQL database (`project_{uuid}`), created dynamically on first access. The main database holds instance-wide data (users, config, projects), while all project-specific data — visualisations, reports, module results, AI interpretations — lives in its own database. This architecture makes cross-tenant data leakage structurally impossible at the database level.

Input Validation & API Security

Route bodies are parsed using typed TypeScript definitions, with Zod available for schema validation. CORS is configured via a `CLIENT_ORIGIN` environment variable with credentials enabled and specific allowed methods and headers. Sensitive headers such as `authorization` and `cookie` are excluded from request logging.

File Uploads

File uploads use the TUS resumable upload protocol. Uploads are written to a temporary directory with UUIDs and moved atomically to the assets directory on completion. Uploads older than 24 hours are automatically cleaned up, and chunk integrity is validated against offset and length headers.

R Module Execution

Data processing modules run in ephemeral Docker containers spawned by the application server (Docker-in-Docker). Each container mounts only the project's sandbox directory, is removed automatically after execution (`--rm`), and runs the R script in an isolated working directory. This limits filesystem exposure to the project sandbox only.

Secrets Management

All secrets (Clerk keys, database passwords, Anthropic API key) are passed to containers via environment variables at runtime and are never hardcoded in the image. A startup validation check ensures all required environment variables are present before the server starts. Secrets are used only server-to-server and are never exposed to the client.

SSL & Infrastructure

TLS termination is handled at the infrastructure level by nginx, with SSL certificates provisioned and managed per instance via certbot. The application server itself runs on HTTP internally behind the reverse proxy. SSH access to the Digital Ocean droplet is restricted to two authorised users via key-based authentication, eliminating password-based login vectors.

Backups

The platform protects against data loss through two independent backup layers. At the application level, automated `pg_dump` snapshots are taken for every instance every 30 minutes and retained for 3 days, giving a fine-grained rolling window for recovering from accidental deletions, corruption, or bad data imports. At the infrastructure level, full Digital Ocean Block Storage volume snapshots are captured on a rolling schedule with daily, weekly, and monthly retention, providing a complete disaster-recovery path covering databases, assets, sandbox, and logs. On-demand backups can also be triggered from within the app through an external status-api service, which stores compressed dumps outside the application container.

Access is gated by the permissions model — `can_create_backups` is required to trigger a backup and `can_restore_backups` to download or restore one (and is blocked entirely for locked projects) — and all backup API calls require both a valid Clerk session and a server-held status-api key. The restore flow validates paths against directory traversal, terminates existing connections, and drops and recreates the target database before loading the dump, ensuring backups provide a reliable recovery path without becoming an attack surface.